



Getting Started with Research Computing

From Command Line to Job Scheduling



1

Meet the User Support Team



Layla
Freeborn



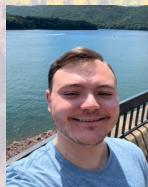
Brandon
Reyes



Andy
Monaghan



Michael
Schneider



John
Reiland



Dylan
Gottlieb



Mohal
Khandelwal



Ragan
Lee



2

2

Website: <https://www.colorado.edu/rc>
Documentation: <https://curc.readthedocs.io>
Helpdesk: rc-help@colorado.edu

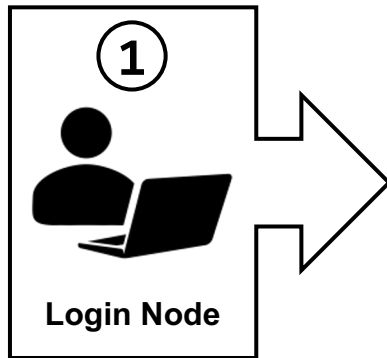


Ask Questions

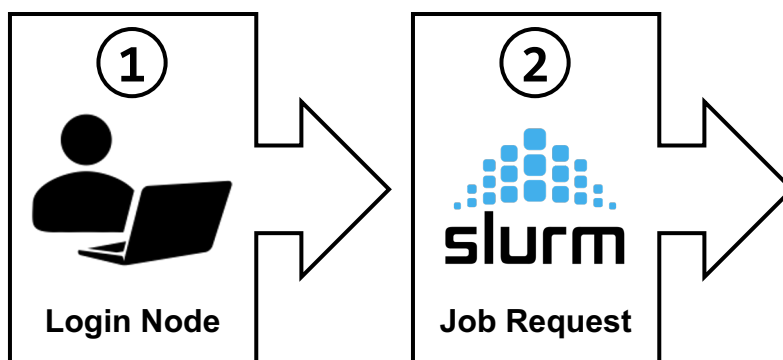


Discuss Ideas

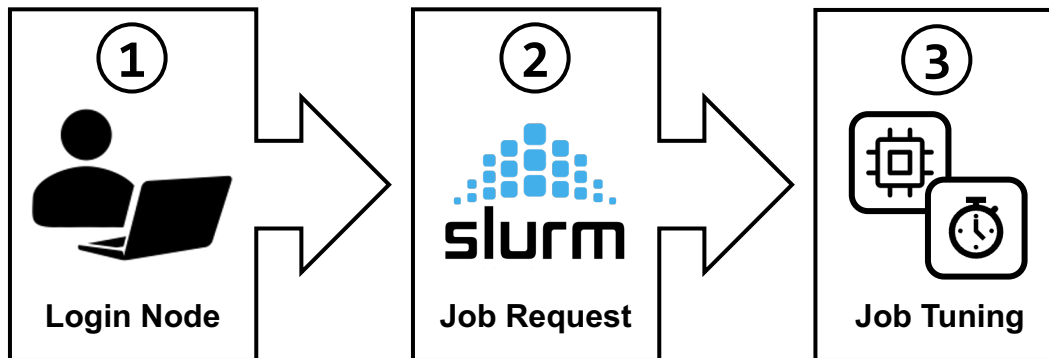
General Overview of Job Submission



General Overview of Job Submission



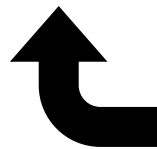
General Overview of Job Submission



Accessing the Terminal

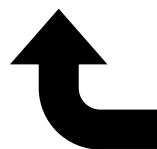


Accessing the Terminal



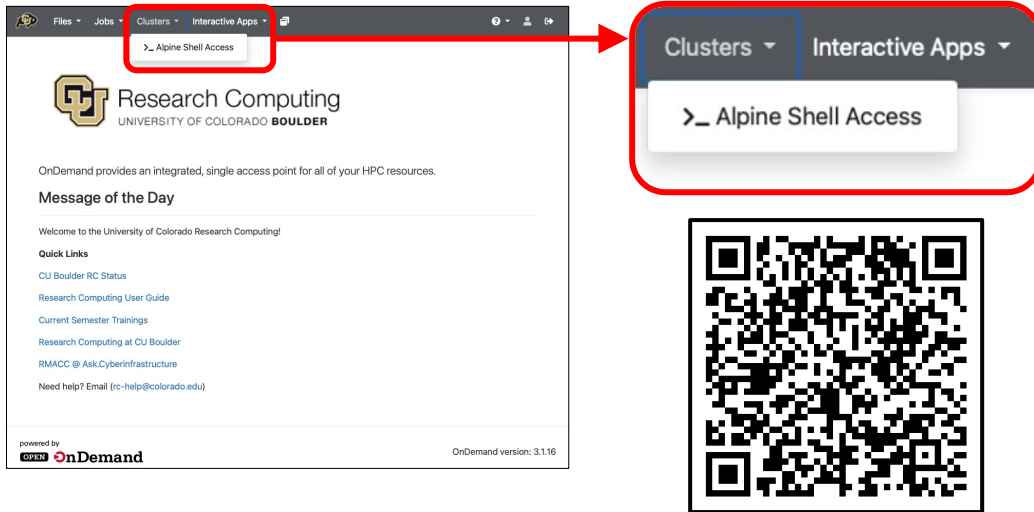
**Restricted – CU Boulder, CSU,
CU Anschutz**

Accessing the Terminal



**Restricted – CU Boulder, CSU,
CU Anschutz**

Demo – Accessing the Alpine Shell



The screenshot shows the University of Colorado Research Computing OnDemand portal. The 'Clusters' menu is highlighted with a red box, and a red arrow points to a larger, detailed view of the 'Clusters' dropdown menu. In this menu, the 'Alpine Shell Access' option is highlighted with a red box. Below the menu view is a QR code for mobile access.

Research Computing
UNIVERSITY OF COLORADO BOULDER

OnDemand provides an integrated, single access point for all of your HPC resources.

Message of the Day

Welcome to the University of Colorado Research Computing!

Quick Links

- [CU Boulder RC Status](#)
- [Research Computing User Guide](#)
- [Current Semester Trainings](#)
- [Research Computing at CU Boulder](#)
- [RMACC @ Ask.Cyberinfrastructure](#)
- [Need help? Email \(rc-help@colorado.edu\)](#)

powered by  OnDemand

OnDemand version: 3.1.16

[Online Documentation](#)

11



Linux Command Line



File Operations



Scripting

12

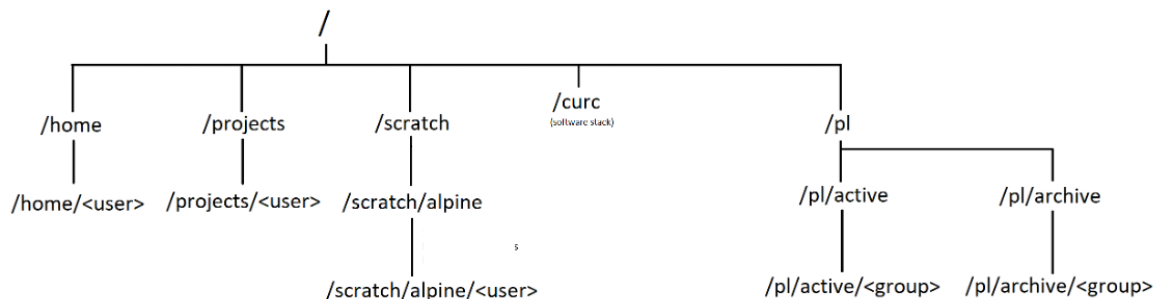
List Files and Directories

ls [option] [file/directory]

- l (long format)
- h (readable file size)
- S (sort by file size)
- t (sort by modification time)
- a (all files)
- r (reverse sort)

13

Your personal directories on CURC



14

File Operations

Change Directory

```
cd [-o] <path>
```

File Operations

Change Directory

```
cd [-o] <path>
```

Make Directory

```
mkdir [-o] <path>
```

File Operations

Change Directory

```
cd [-o] <path>
```

Make Directory

```
mkdir [-o] <path>
```

Copy / Move Files & Directories

```
cp [-o] <source> <dest>
```

File Operations

Change Directory

```
cd [-o] <path>
```

Make Directory

```
mkdir [-o] <path>
```

Copy / Move Files & Directories

```
cp [-o] <source> <dest>
```

Delete Files

```
rm [-o] <path>
```

Create a file – Terminal Text Editors



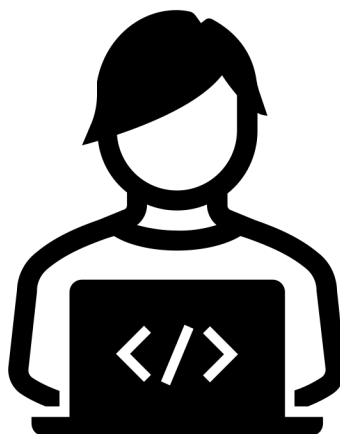
Vim

```

:::
iLE88Dj. :jd88888Dj:
.LGitE888D.f86jjjL8888E;
iE :8888Et. .G8888.
;i E888, ,8888,
D888, :8888:
D888, :8888:
D888, :8888:
D888, :8888:
888W, :8888:
W88W, :8888:
W88W, :8888:
DGGD: :8888:
:8888:
:W888:
:8888:
E888i
tw88D
  
```

Nano

Demo – Creating a Bash Script



Shell script basics

- In shell scripts, the first line must specify the shell e.g. `#!/bin/bash`
- The program loader recognizes the `#!` and will interpret the rest of the line (`/bin/bash`) as the interpreter program.
- If a line starts with `#`, it is a comment and is not run.

```
#!/bin/bash
```

Shell to run

```
# the files in /tmp.
```

Comment

```
cd /tmp
```

Change directories

```
ls
```

List everything in /tmp

Run test.sh

```
#!/bin/bash
```

```
# the files in /tmp.
```

```
cd /tmp
```

```
ls
```

chmod u+x test.sh
./test.sh

Modes (aka permissions)

- View file/directory permissions `ls -l`
- 3 classes of users:
 - User (u) aka “owner”
 - Group (g)
 - Other (o)
- 3 types of permissions:
 - Read (r)
 - Write (w)
 - Execute (x)

Diagram illustrating the permissions string `-rwxr-xr--` with labels:

- `-`: File type (indicated by a bracket labeled "directory")
- `rwx`: Permissions for the user (indicated by a bracket labeled "user")
- `r-x`: Permissions for the group (indicated by a bracket labeled "group")
- `r--`: Permissions for other (indicated by a bracket labeled "other")

Modes (continued)

`chmod` changes modes:

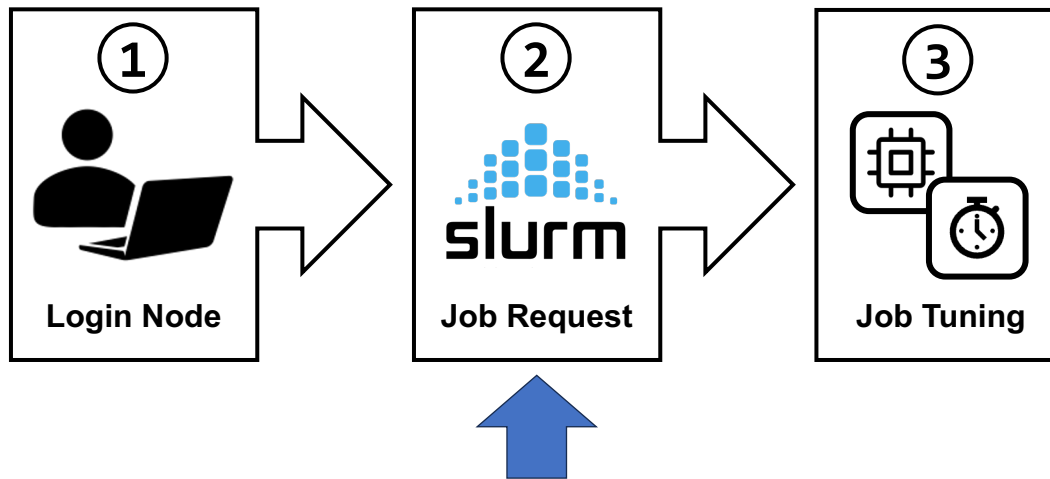
To add write and execute permission for your group:

`chmod g+wx filename`

To remove execute permission for others:

`chmod o-x filename`

General Overview of Job Submission



Interactive jobs

acompile

- Quick access to resources
- Limit of 4 CPUs for up to 12 hours
- Focused on compiling/building software and small workflows

sinteractive

- Request any partition, more resources, and longer walltime
- Subject to standard wait times (not immediate access)
- As the name implies, allows you to access a job interactively

Running an interactive job

- Here we will run a Python script using an interactive job
- First request resources:

```
$ sinteractive --partition=atesting --qos=testing --time=00:10:00 --ntasks=1
```

- When the job starts you will be put on a compute node and you can execute your commands:

```
$ module load anaconda
$ python my_very_cool_script.py
```

- To quit:

```
$ exit
```

Batch Jobs

- **Batch Jobs** are jobs you submit to the scheduler that are run later without supervision
- A job script is simply a script that includes **SLURM directives** (resource specifics) ahead of any commands.

Anatomy of a job script

```
#!/bin/bash

## Directives
#SBATCH --<option>=<value>

## Software
module load <software>

## User scripting
<command>
```

Directives in a job script

```
#SBATCH --<option>=<value>
```

Directives in a job script

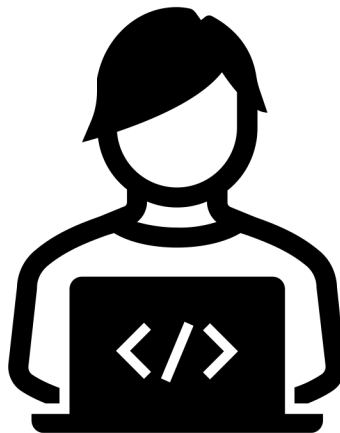
```
#SBATCH --<option>=<value>
```



**Options listed
at the end**

31

Demo – Creating a *Batch* Script



32

Example job script

```
#!/bin/bash

## Directives
#SBATCH --ntasks=1           # Number of requested tasks/cores
#SBATCH --time=0:01:00       # Max run time
#SBATCH --partition=amilan    # Specify Alpine CPU node
#SBATCH --qos=normal          # Specify QoS
#SBATCH --output=test_%j.out  # Rename standard output file

## Software
module purge                  # Purge all existing modules

## User commands
echo "This is a test submitted by $USER"
```

Submitting a Job script

`sbatch <job_file> <other-directives>`

- Command to submit a job to the SLURM job scheduler

Submitting a Job script

`sbatch <job_file> <other-directives>`

- Command to submit a job to the SLURM job scheduler
- If we created the job script “my_first_job.sh” then we would submit it as follows:

`sbatch /path/to/my_first_job.sh`

Submitting a Job script

`sbatch <job_file> <other-directives>`

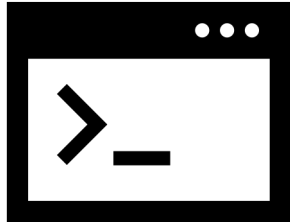
- Command to submit a job to the SLURM job scheduler
- If we created the job script “my_first_job.sh” then we would submit it as follows:

`sbatch /path/to/my_first_job.sh`

- Modify slurm directives in the batch script:

`sbatch /path/to/my_first_job.sh --ntasks=12`

Local vs Environment variables



Environment
Accessible across
scripts / shells



Local
Only exists in
current script / shell

Environment variables

- Environment variables store important information needed by Linux users and programs
- Type `env` to see your currently set up environment variables
- Useful environment variables:

<code>PATH</code>	directories to search for commands
<code>HOME</code>	home directory
<code>PWD</code>	current working directory
<code>USER</code>	username
<code>LD_LIBRARY_PATH</code>	directories to search for dynamically-loaded libraries

Local vs Environment Variables

```
[username@login-ci3 rc_temp] export g="Hi!"
```

```
#!/bin/bash  
# set local variable  
a="bb"  
# print out the inherited variable  
echo $g
```

Local vs Environment Variables

```
[username@login-ci3 rc_temp] ./test.sh
```

```
[username@login-ci3 rc_temp] echo $a
```

Batch vs Interactive



**Batch
Job**



**Interactive
Job**

Batch vs Interactive

- Runs a job script
- No need for user input
- Best for tested workflows and long/big jobs



**Batch
Job**



**Interactive
Job**

Batch vs Interactive

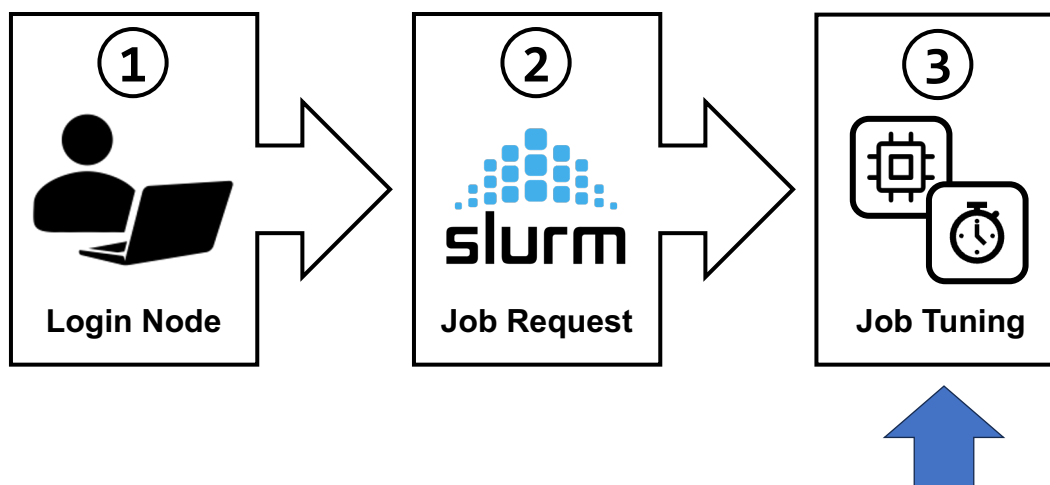
- Runs a job script
- No need for user input
- Best for tested workflows and long/big jobs



- Terminal or GUI
- Requires active user input
- Best for development and exploration

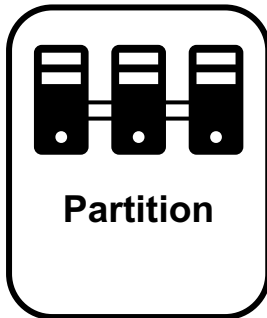
43

General Overview of Job Submission

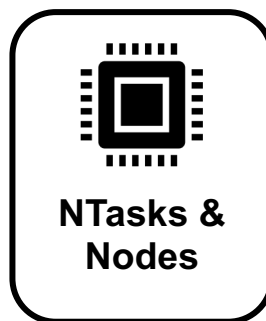
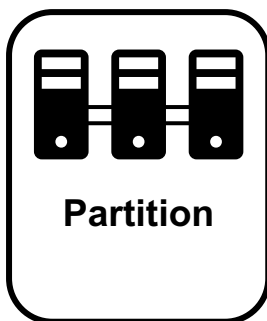


44

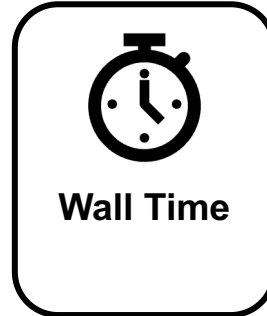
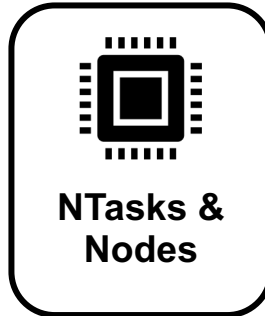
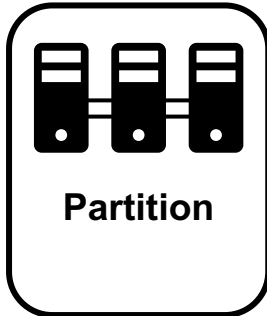
Common Resource Requests



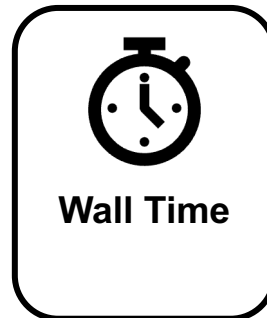
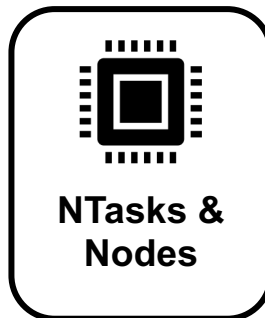
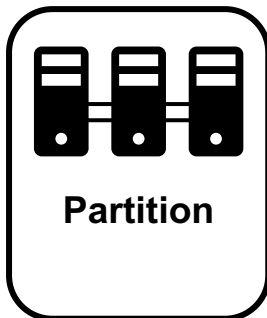
Common Resource Requests



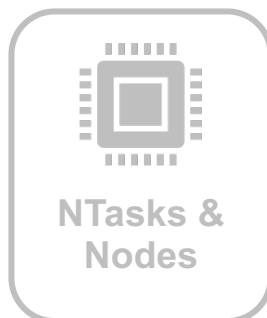
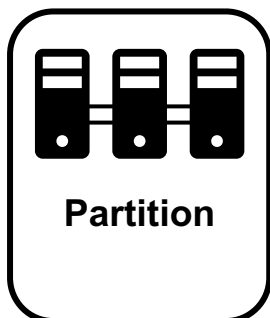
Common Resource Requests



Common Resource Requests

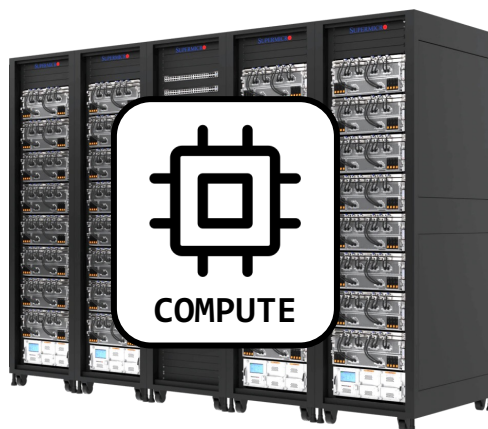


Common Resource Requests



49

Alpine Partitions



50

Alpine Partitions

amilan
General Usage



Alpine Partitions

amilan
General Usage

amem
High Memory



Alpine Partitions

amilan

General Usage

amem

High Memory



aa100

Nvidia GPU's

Alpine Partitions

amilan

General Usage

amem

High Memory



aa100

Nvidia GPU's

ami100

AMD GPU's

Alpine Partitions

- Partitions are a collection of compute nodes e.g. computers with common characteristics

Partition	Description	# of nodes	RAM/core (GB)	cores/node	GPUs/node
amilan	General Compute Node: AMD Milan	406	3.75	32,48,64	0
ami100	GPU Node: 3x AMD MI100	8	3.75	64	3
aa100	GPU Node: 3x Nvidia A100	12	3.75	64	3
amem	High-memory node	24	21.5	48,64	0

Alpine Testing Partitions

- For many of the partitions we have testing partitions that provide quicker access to resources.

Note: there are restrictions on the amount of resources you can request. These restrictions can be found in our documentation.

Partition	Description	# of nodes	RAM/core (GB)	cores/node	GPUs/node
atesting	General Compute Node: AMD Milan	2	3.75	64	0
acompile	General Compute Node: AMD Milan	4	3.75	64	0
atesting_mi100	GPU Node: 3x AMD MI100	1	3.75	64	3
atesting_a100	GPU Node: 3x Nvidia A100	1	3.75	64	6 MIG instances

Quality of Service (QoS)

- Quality of Service specifies additional constraints for a job
 - On Alpine, QoS can be used to run long jobs, specify testing partitions, and select high-memory nodes

QoS	Description	Max wall time	Max jobs/user	Max nodes/user
normal	Default QoS	24 H	1000	128
long	For jobs needing longer wall times	7 D	200	20
mem	High-memory jobs	7 D	n/a	12
testing	For jobs submitted to testing partitions	1 H	1	n/a
compile	For jobs submitted to compile partition	12 H	1	n/a

Common directives

- ★ • **Partition** – A collection of compute nodes
 - `--partition=<partition_name>`
 - ★ • **Quality of service (QoS)** – System defined constraints for a job (more on this later!)
 - `--qos=<qos>`
 - **Allocation** – Account to “charge to”
 - `--account=<account_name>`
 - **Number of nodes to run on**
 - `--nodes=<nodes>`
 - **Number of cores to run on**
 - `--ntasks=<number-of-tasks>`
- ★ - Must be specified for every job

Common directives

- ★ • **Wall time** – How long you want to run on these resources
 - `--time=<wall time>`
- **Job name**
 - `--job-name=<jobname>`
- **Output** – Where all output that would be written to the terminal should go
 - `--output=<name>`
- **Send an email when events happen in the job**
 - `--mail-type=<type>`
- **Email address to send updates to**
 - `--mail-user=<user>`

★ - Must be specified for every job

Checking your jobs

- **squeue**: Monitor your jobs status **in queue and while running**:
 - By default, shows all jobs in queue can specify using:

```
$ squeue -u <username>
$ squeue -p <partition>
```

- **sacct**: Check back on usage statistics of **previous Jobs**
 - By default, only checks all jobs from the start of the current day can specify using:

```
$ sacct -u <username>
$ sacct --start=MM/DD/YY -u <username>
$ sacct -j <job-id>
```

Checking your jobs

- To check the percentage of CPU and memory usage of a job **after it completes**, use **seff**

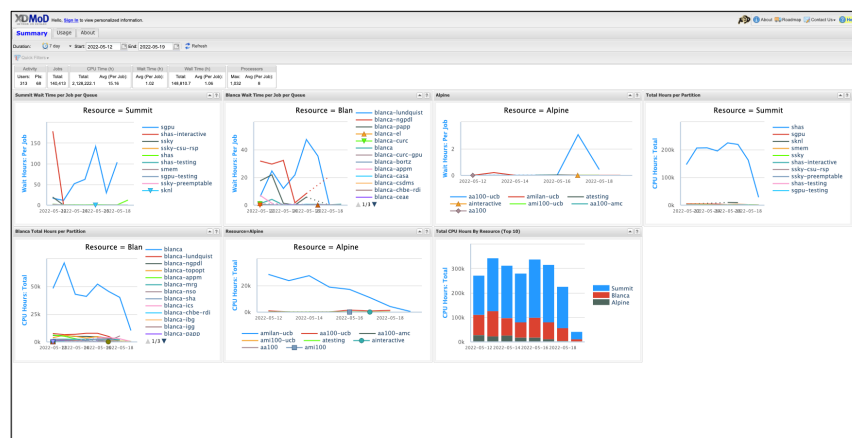
```
$ module load slurmtools
$ seff <job number>
```

- To see general statistics for your submitted jobs, use **jobstats**

```
$ module load slurmtools
$ jobstats <username> <# of days>
```

61

Job Monitoring - XDMOD



62

Thank you!

- **Documentation:** curc.readthedocs.io/
- **Trainings with Center for Research Data and Digital Scholarship (CRDDS):**
<https://www.colorado.edu/crdds/>
- **Helpdesk:** rc-help@colorado.edu
- **Consult Hours** (Tuesday 12:00-1:00 in-person, Thursday 1:00-2:00 virtually)

Survey and feedback

<https://tinyurl.com/curc-survey18>

